目录

Billing State Machine（计费状态机）

一、钱包余额状态

状态约束

余额组成

二、单次请求的完整生命周期

2.1 非流式请求

2.2 流式请求

三、Freeze / Settle / Cancel 详细规格

3.1 Freeze（预冻结）

3.2 Settle（结算）

3.3 Cancel（退还）

四、赠送额度扣费优先级

五、幂等性状态机

六、并发控制

6.1 PostgreSQL 层面

6.2 Redis 层面

6.3 冻结过期

Billing State Machine（计费状态机）

> **版本**：v2.0 **日期**：2026-06-18 **用途**：开发团队实现计费模块的核心参考 **变更**：基于 v1.0 修正，补充幂等性状态机、赠送额度扣费优先级、并发控制细节

一、钱包余额状态

状态约束

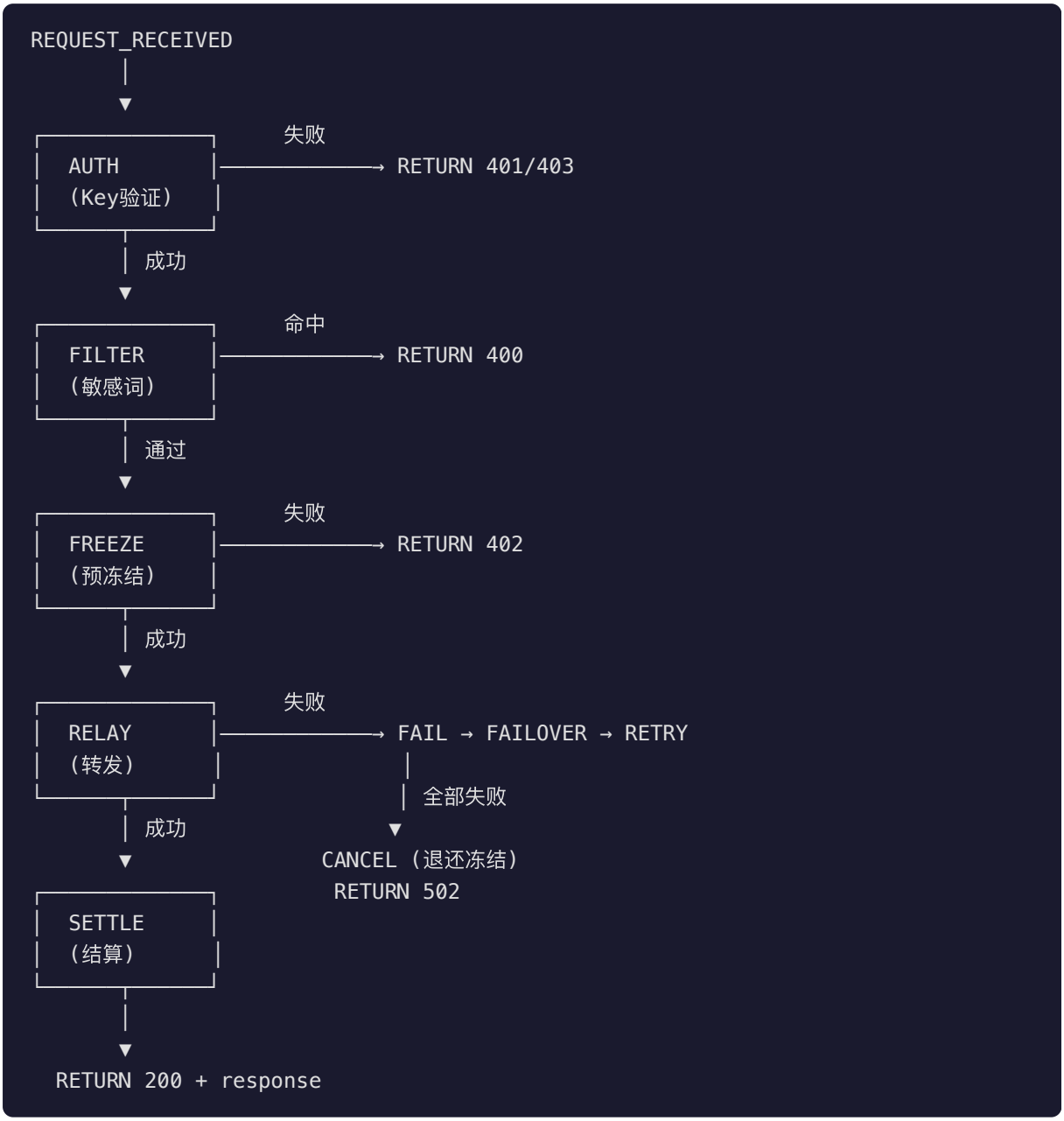
```
ALTER TABLE wallets ADD CONSTRAINT wallets_balance_check CHECK (balance >= 0);
ALTER TABLE wallets ADD CONSTRAINT wallets_frozen_check CHECK (frozen >= 0);
ALTER TABLE wallets ADD CONSTRAINT wallets_granted_check CHECK (granted_balance >=
```

余额组成

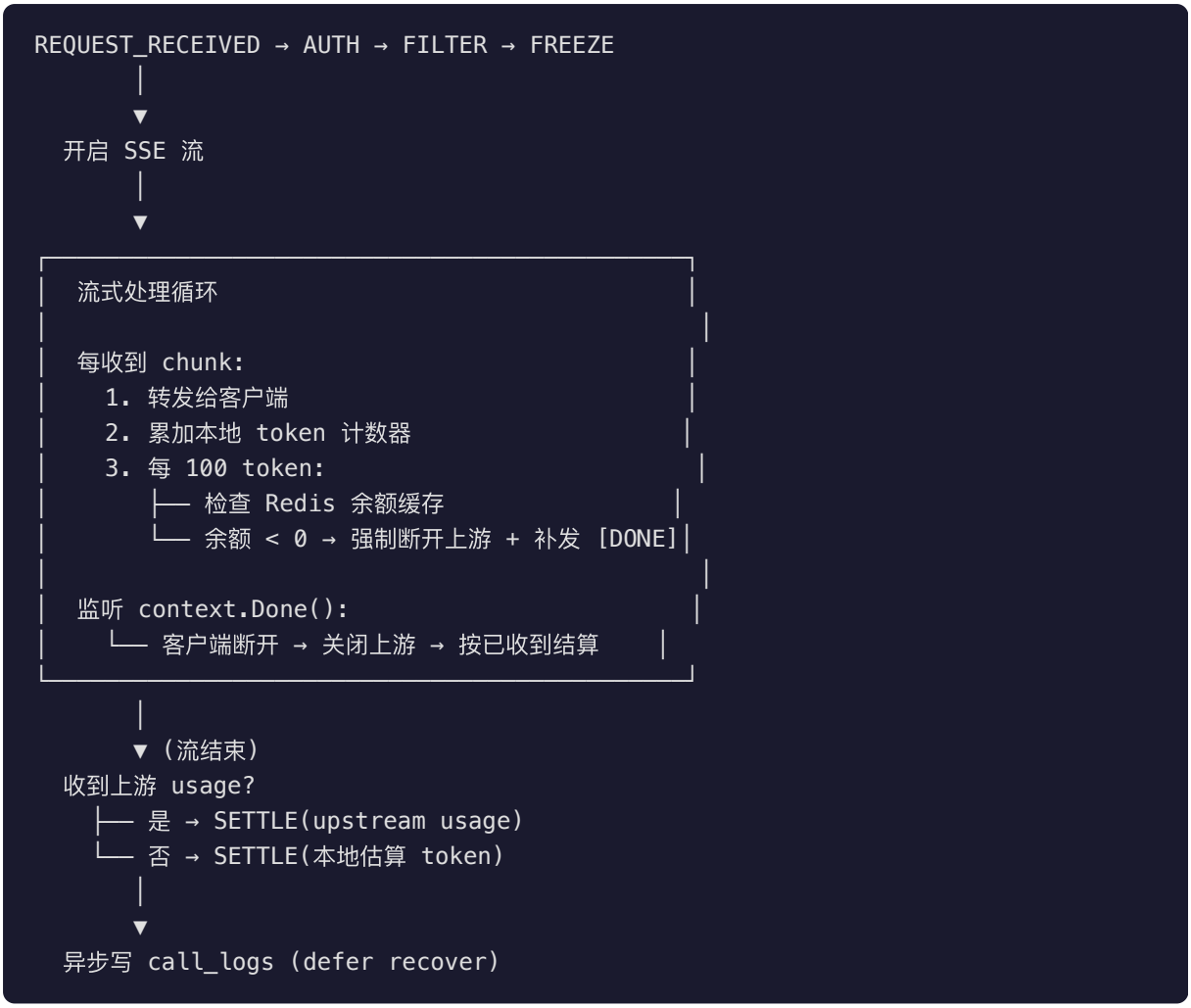
```
┌──────────────────────────────────────────────┐
│                  wallets 表                  │
│                                              │
│  balance: ¥100.00          (充值余额)        │
│  granted_balance: ¥20.00   (赠送余额，有过期时间)  │
│  frozen: ¥0.00             (当前冻结金额)    │
│  total_spent: ¥50.00       (累计消费)        │
│                                              │
│  可用余额 = balance + granted_balance        │
└──────────────────────────────────────────────┘
```

二、单次请求的完整生命周期

2.1 非流式请求



2.2 流式请求



三、Freeze / Settle / Cancel 详细规格

3.1 Freeze（预冻结）

```
// 伪代码
func Freeze(orgID int64, freezeAmount int64, requestID string) error {
    // 1. 幂等性检查
    existing := db.Query("SELECT status FROM freeze_idempotency WHERE request_id =
    if existing.Status == "Success" {
        return nil // 直接返回
    }
    if existing.Status == "Processing" {
        return ErrConflict // 409
    }

    // 2. 插入幂等性记录
    db.Exec("INSERT INTO freeze_idempotency (request_id, status) VALUES (?, 'Proce

    // 3. Redis 快速拒绝
    cachedBalance := redis.Get("balance:" + orgID)
    if cachedBalance < freezeAmount {
        return ErrInsufficientBalance // 402
    }

    // 4. PostgreSQL 原子冻结
    result := db.Exec(`
        UPDATE wallets
        SET balance = balance - ?,
            frozen = frozen + ?
        WHERE org_id = ?
          AND (balance + granted_balance) >= ?
    `, freezeAmount, freezeAmount, orgID, freezeAmount)

    if result.RowsAffected() == 0 {
        // 5. 赠送余额兜底
        result = db.Exec(`
            UPDATE wallets
            SET granted_balance = granted_balance - ?,
                frozen = frozen + ?
            WHERE org_id = ?
              AND granted_balance >= ?
        `, freezeAmount, freezeAmount, orgID, freezeAmount)
    }

    if result.RowsAffected() == 0 {
        db.Exec("UPDATE freeze_idempotency SET status = 'Failed' WHERE request_id
        return ErrInsufficientBalance
    }

    // 6. 更新 Redis 缓存
    redis.DecrBy("balance:"+orgID, freezeAmount)

    db.Exec("UPDATE freeze_idempotency SET status = 'Success' WHERE request_id = ?
    return nil
}
```

3.2 Settle（结算）

```
func Settle(orgID int64, freezeAmount int64, actualCost int64, requestID string) {
    refund := freezeAmount - actualCost

    if refund > 0 {
        // 退还差额
        db.Exec(`
            UPDATE wallets
            SET frozen = frozen - ?,
                balance = balance + ?
            WHERE org_id = ?
        `, freezeAmount, refund+actualCost, freezeAmount)
    } else {
        // 冻结金额全部消费
        db.Exec(`
            UPDATE wallets
            SET frozen = frozen - ?,
                total_spent = total_spent + ?
            WHERE org_id = ?
        `, freezeAmount, actualCost, orgID)
    }

    // 写交易记录
    db.Exec(`
        INSERT INTO transactions (request_id, org_id, type, status, amount)
        VALUES (?, ?, 'settle', 'completed', ?)
    `, requestID, orgID, actualCost)
}
```

3.3 Cancel（退还）

```
func Cancel(orgID int64, freezeAmount int64, requestID string) {
    db.Exec(`
        UPDATE wallets
        SET frozen = frozen - ?,
            balance = balance + ?
        WHERE org_id = ?
    `, freezeAmount, freezeAmount, orgID)

    db.Exec(`
        INSERT INTO transactions (request_id, org_id, type, status, amount)
        VALUES (?, ?, 'cancel', 'completed', ?)
    `, requestID, orgID, freezeAmount)
}
```

四、赠送额度扣费优先级

```
扣费顺序：
1. granted_balance（按过期时间近到远）
   └── 过期的 granted 自动清零（后台定时任务）
2. balance（充值余额）

示例：
  granted_balance: ¥50（其中 ¥20 下周过期，¥30 下月过期）
  balance: ¥100
  冻结金额: ¥80

  扣费：
  1. 先扣 granted_balance ¥20（即将过期的）
  2. 再扣 granted_balance ¥30（下月过期的）
  3. 最后扣 balance ¥30
  4. 剩余：balance = ¥70, granted_balance = ¥0
```

五、幂等性状态机

```
freeze_idempotency 表：

  request_id (PK)
  status: Processing → Success / Failed
  created_at
  updated_at

状态转换：
  Processing → Success（正常完成）
  Processing → Failed（余额不足/系统错误）
  Processing → Timeout（created_at + 300s 无更新，可重新处理）

超时处理：
  后台定时任务每 60s 扫描 Processing 状态超过 300s 的记录
  标记为 Timeout，允许客户端重试
```

六、并发控制

6.1 PostgreSQL 层面

- `WHERE (granted_balance + balance) >= amount` 行锁

- 同一 org_id 的并发请求串行化（PostgreSQL 行级锁）
- 不同 org_id 的请求完全并行

6.2 Redis 层面

- `DECRBY balance:{orgID} amount` 原子操作
- 快速拒绝明显不足的请求（减少 PG 压力）
- 余额缓存与 PG 最终一致（结算后更新）

6.3 冻结过期

- 后台定时任务每 60s 扫描 `frozen > 0` 且无对应 `call_logs` 的记录
- 超过 300s 无结算的冻结自动退还
- 防止进程崩溃导致的”僵尸冻结”